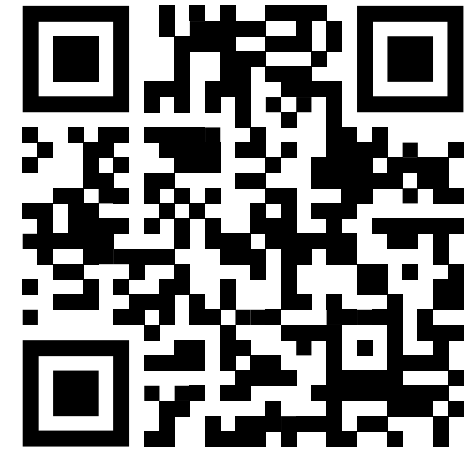


Programmierung in C++

Klassenbeziehungen

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Vinzenz Funk



<https://poll.hs-kempten.de/poll/>

Klassenbeziehungen



Einführung

- Neben Vererbungen (Erweiterung einer Klasse), sind Beziehungen zwischen unterschiedlichen Klassen ein wichtiger Bestandteil der Softwareentwicklung
- Klassenbeziehungen sind allgemein – unabhängig von der verwendeten Programmiersprache – definiert
 - Häufig als **UML-Klassendiagramm** dargestellt
- Klassenbeziehungen beruhen stark auf Prinzipien der Objektorientierung
 - **Polymorphismus**: Objektdaten werden vererbt und überschrieben
 - **Abhängigkeit & Assoziation**: Objekte stehen in Beziehung zueinander
 - **Aggregation & Komposition**: Objekte setzen sich aus Teilobjekten zusammen
- Alle diese Prinzipien können in C++ realisiert werden
 - Dabei gibt es im Vergleich zu anderen objektorientierten Sprachen jedoch Besonderheiten zu beachten



Beziehungstypen

Lernziele

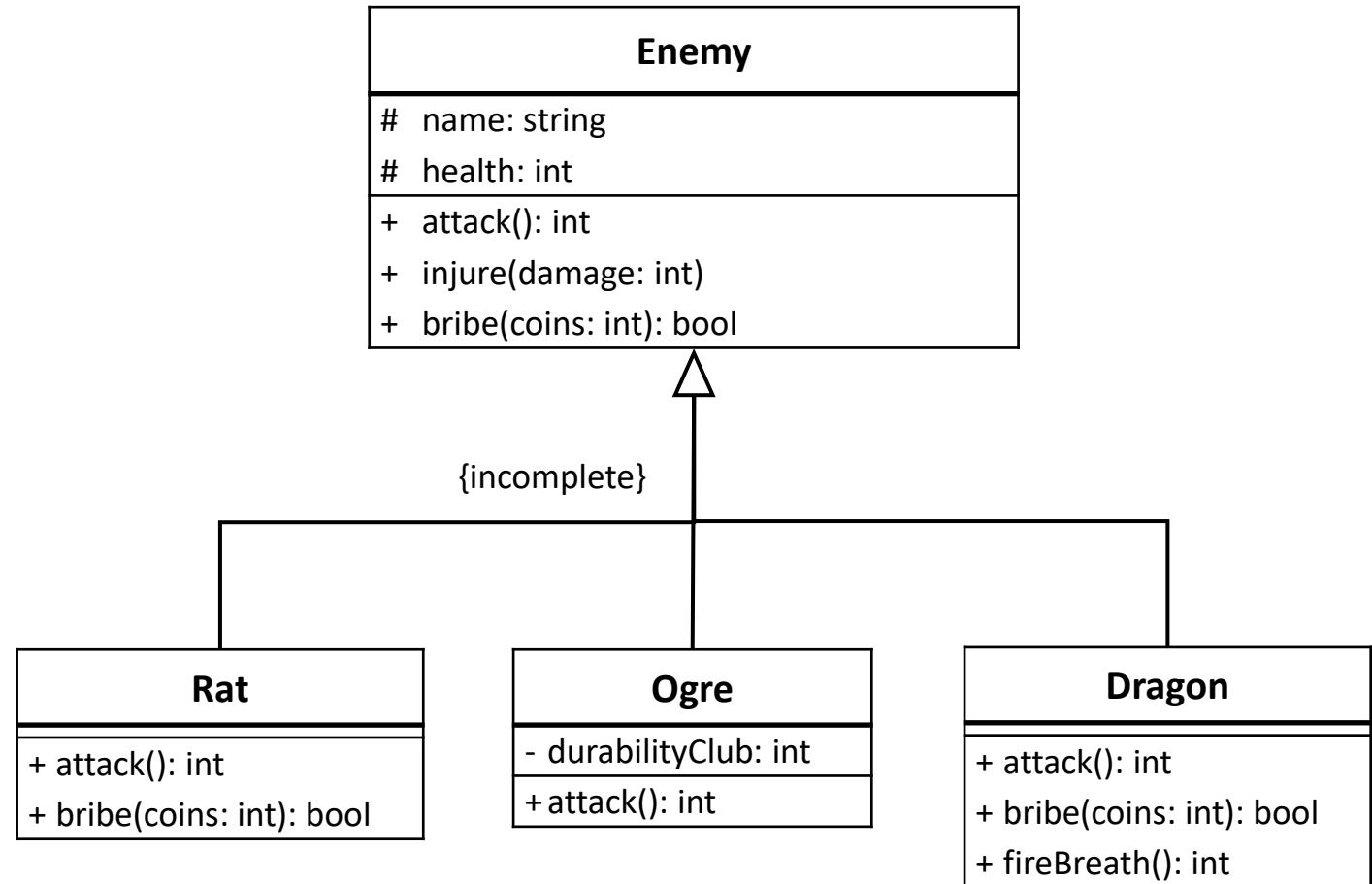
- Das Wissen über Klassenbeziehungen wurde um Abhängigkeiten, Assoziationen, Aggregationen und Kompositionen erweitert
- Allgemein-definierte Klassenbeziehungen können in C++-Code übersetzt und implementiert werden



Beziehungstypen

Klassendiagramme bisher

- Bisher kennen wir bei UML-Klassendiagrammen:
 - **Klassendefinitionen** (Rechteck mit Name, Attribute, Methoden)
 - **Sichtbarkeiten** (+public, #protected, -private)
 - **Vererbungen** (Linie mit nicht-ausgefülltem Dreieck)
 - **Properties** (Zusätze auf Relationslinien)
- Folgend werden weitere Elemente für Beziehungen zwischen Klassen eingeführt...

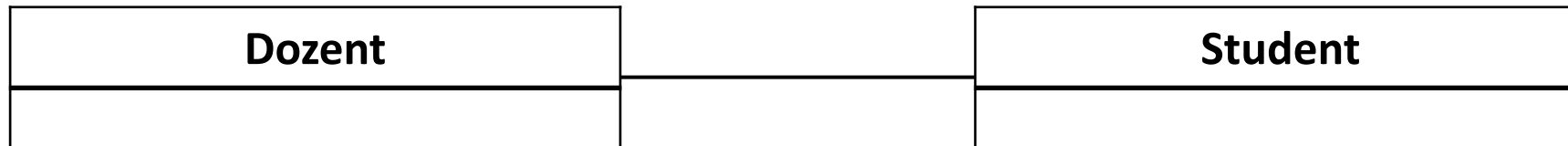


Beziehungstypen



Darstellung

- Eine Beziehung zwischen zwei Klassen wird mit einer **Verbindungsline** zwischen den Klassen dargestellt
 - Die Art der Verbindungsline (gestrichelt, durchgezogen, Symbol am Ende, ...) hängt vom **Beziehungstyp** ab



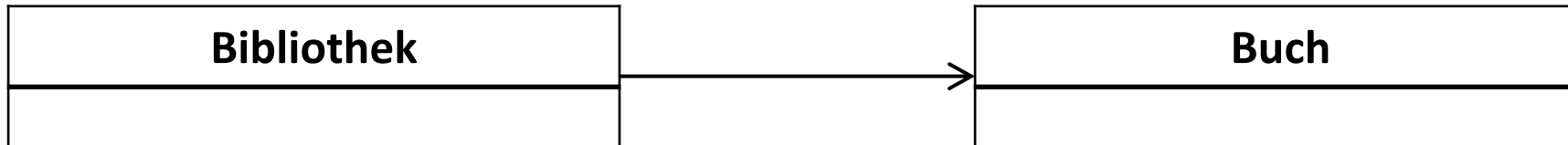
Dozent und Student stehen in Beziehung zueinander

Beziehungstypen



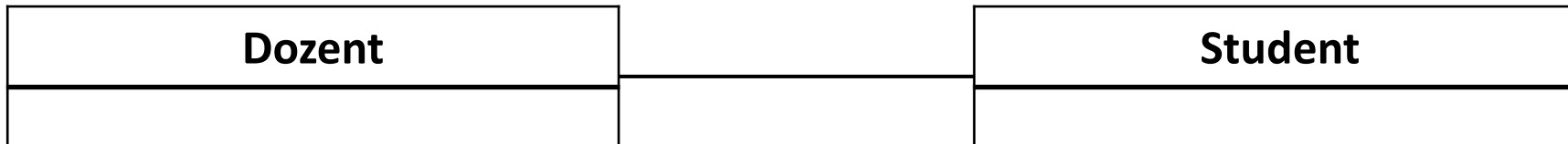
Richtung

- Eine Beziehung kann **gerichtet** oder **ungerichtet** sein
 - **gerichtet** (unidirektional): Eine Klasse steht in Beziehung zu einer anderen Klasse, aber nicht umgekehrt. Dargestellt durch **nicht-ausgefüllten Pfeil** am Ende der Verbindungslinie:



Bibliothek steht in Beziehung zu Buch
Buch steht nicht in Beziehung zu Bibliothek

- **ungerichtet** (bidirektional): Die Klassen stehen beide in Beziehung zueinander. Dargestellt **ohne Pfeil** am Ende der Verbindungslinie:



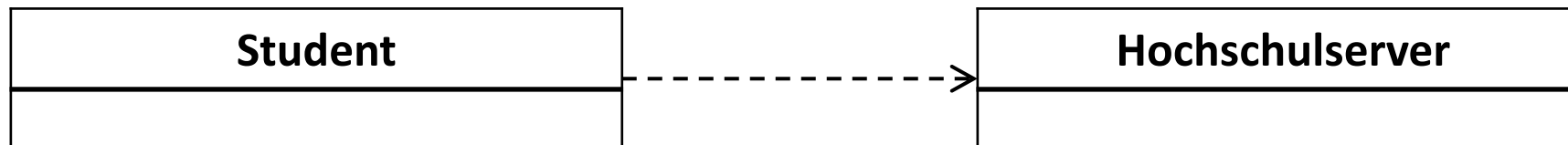
Dozent steht in Beziehung zu Student
Student steht in Beziehung zu Dozent

Beziehungstypen



Abhängigkeit

- Eine **Abhängigkeits**beziehung stellt bei UML-Klassendiagrammen eine schwache Beziehung zwischen zwei Klassen dar
 - Die Klassen kennen sich für einen kurzen Zeitraum und interagieren miteinander
 - Abhängigkeiten werden häufig für **Parameter**, **Rückgabewerte** oder **lokale Variablen** einer Methode modelliert
- Abhängigkeitsbeziehungen werden mit einer **gestrichelten Linie** dargestellt



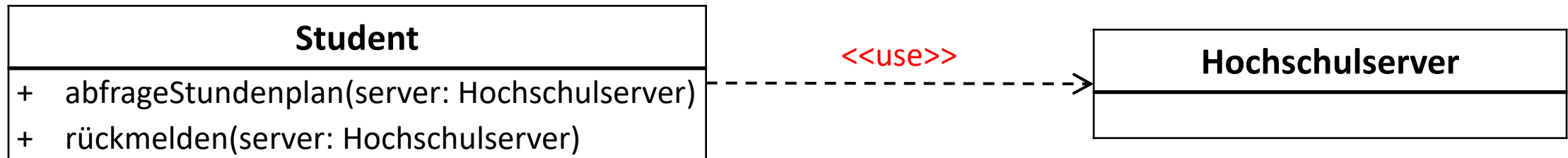
Student kennt für einen kurzen Zeitraum Hochschulserver

Beziehungstypen



Abhängigkeit – Stereotyp

- Über einen **Stereotyp** wird angegeben, in welcher Art der Abhängigkeit die Klassen miteinander stehen
 - Der Stereotyp wird zwischen **zwei spitzen Klammern** über der Beziehungslinie dargestellt



*Student **nutzt** (<<use>>) für einen kurzen Zeitraum Hochschulserver*

Durch die Methodenparameter entsteht eine kurzfristige Beziehung zum Hochschulserver

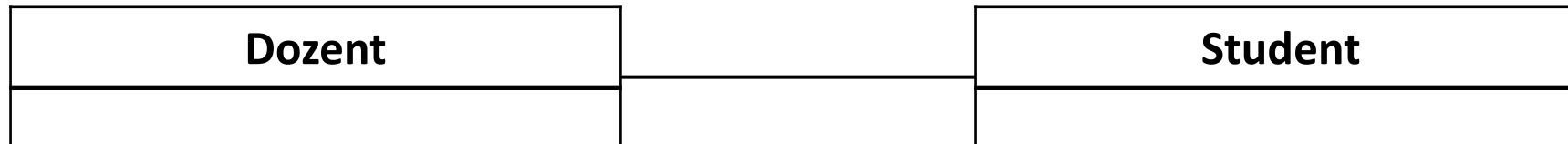
- Abhängigkeitsbeziehungen werden häufig zur besseren Verständlichkeit eines UML-Klassendiagramms angegeben – sie werden nicht als Membervariablen o.Ä. umgesetzt

Beziehungstypen



Assoziation

- Eine **Assoziation** stellt bei UML-Klassendiagrammen eine starke Beziehung zwischen zwei Klassen dar
 - Die Klassen kennen sich über einen längeren Zeitraum hinweg und interagieren miteinander
 - Assoziationen werden häufig für **Membervariablen** modelliert
- Assoziationen werden mit einer **durchgezogenen Linie** dargestellt
 - Es existieren keine Symbole an den Enden der Linie (außer Pfeil bei gerichteter Assoziation)



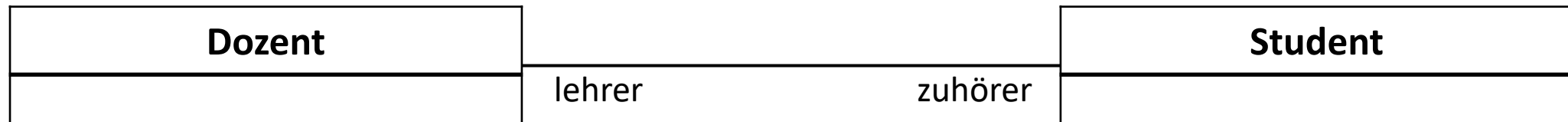
*Dozent kennt Student langfristig
Student kennt Dozent langfristig*

Beziehungstypen



Assoziation – Rollen

- Bei einer Assoziation kann weiter angegeben werden, welche **Rolle** eine Klasse im Bezug zur anderen Klasse einnimmt
 - Rollennamen werden an das Ende einer Assoziationslinie zu der Klasse geschrieben, deren Rolle dargestellt wird



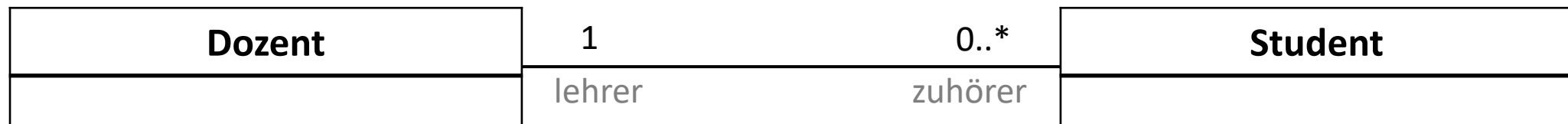
*Ein Dozent hat Student als Zuhörer
Ein Student hat Dozent als Lehrer*

Beziehungstypen



Assoziation – Multiplizitäten

- Neben der Rolle kann bei Assoziationen auch die **Multiplizität** angegeben werden
= mit wie vielen Objekten der gegenüberliegenden Klasse ein Objekt assoziiert ist
 - Multiplizitäten werden – wie auch die Rollen – an das Ende einer Assoziationslinie zu der Klasse geschrieben, deren Multiplizität dargestellt wird



Ein Dozent hat **keine bis beliebig viele [0..*]** Studenten als Zuhörer

Ein Student hat **genau einen [1]** Dozenten als Lehrer

Beziehungstypen



Assoziation – Multiplizitäten

- Multiplizitäten können eine bestimmte Anzahl oder einen Bereich darstellen
 - **Bereiche** werden mit zwei Punkten definiert, an deren Enden die Startanzahl und die Endanzahl steht
 - Mehrere **Anzahlen** können Komma-getrennt angegeben werden
 - Ein **Stern (*)** steht für eine beliebige Anzahl, inklusive null
- Beispiele:

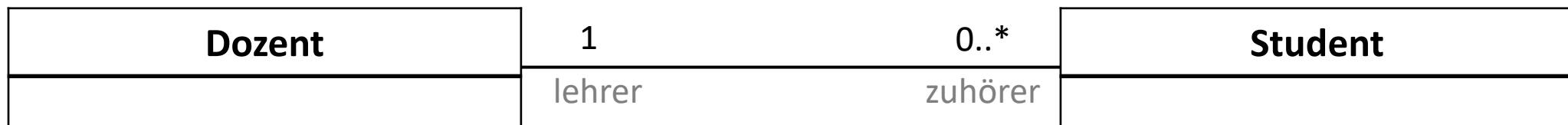
Multiplizität	Aussage
*	Größer oder gleich Null (Standard, wenn nichts angegeben ist)
0..*	Größer oder gleich Null (wie oben)
1	Genau eins
0, 1	Null oder eins
0..3, 7	Null, eins, zwei, drei oder sieben

Beziehungstypen



Assoziation – Implementierung

- Assoziationen können meist direkt als Membervariablen einer Klasse umgesetzt werden
 - Sie tauchen i.d.R. nicht nochmal in der Attributliste der Klassendefinitionen auf!
- Umsetzung vorheriges Beispiel:



```
class Dozent
{
private:
    std::vector<Student*> m_zuhörer;
};
```

```
class Student
{
private:
    Dozent* m_lehrer;
};
```

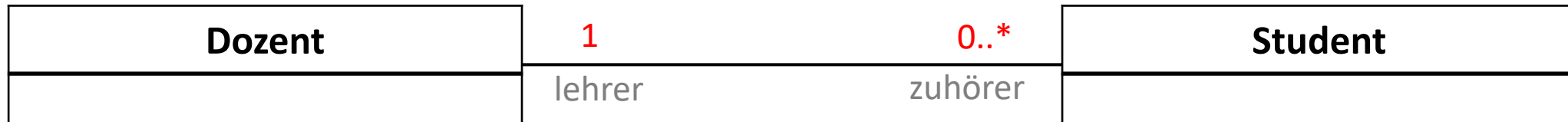
Annahme: Zeiger können nicht Nullpointer sein! (Später muss das programmatisch geprüft werden)
Die Zeiger in diesem und folgenden Beispielen können alle als `std::shared_ptr` umgesetzt werden

Beziehungstypen

Assoziation – Implementierung



- Die Implementierung lässt sich direkt aus dem Klassendiagramm ableiten



```
class Dozent
{
private:
    std::vector<Student*> m_zuhörer;
};
```

`std::vector<Student*>` entspricht 0..* Zuhörern.
Kann auch als andere Containerklasse umgesetzt werden.

```
class Student
{
private:
    Dozent* m_lehrer;
};
```

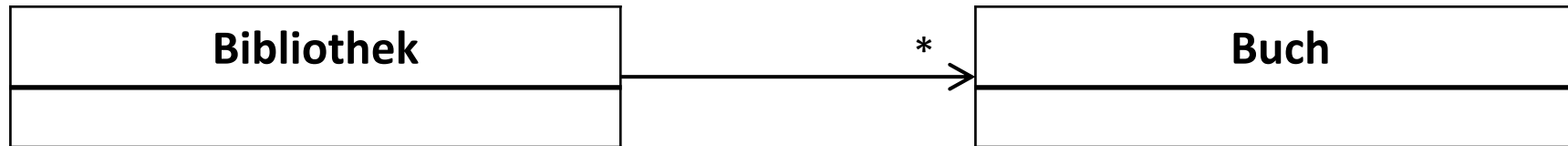
`Dozent*` entspricht 1 Lehrer

Beziehungstypen



Assoziation – Implementierung – Gerichtet

- Bei **gerichteten Assoziationen** wird nur in der Klasse eine Membervariable umgesetzt, von der die Assoziation ausgeht



Eine Bibliothek kennt **beliebig viele** Bücher
Ein Buch kennt **keine** Bibliothek

```
class Bibliothek
{
private:
    std::vector<Buch*> m_bücher;
};
```

```
class Buch
{
};
```

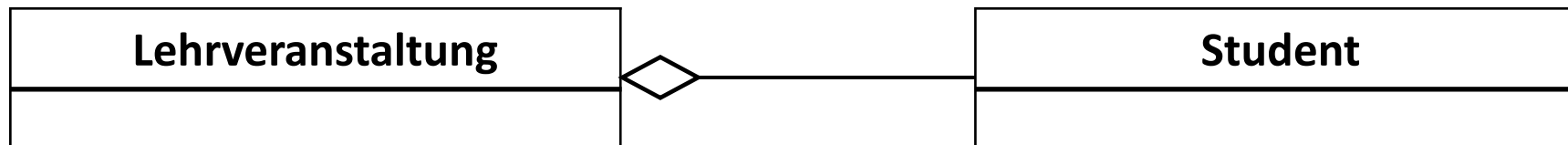
Kennt keine Bibliothek

Beziehungstypen



Aggregation

- Eine **Aggregation** stellt bei UML-Klassendiagrammen eine **Teil-Ganzes-Beziehung** dar
 - Eine Klasse ist ein Teil einer anderen Klasse = des Ganzen
 - Die Einzelteile können auch ohne das Ganze existieren
- Aggregationen werden mit einer durchgezogenen Linie dargestellt, an deren Ende sich eine **nicht-ausgefüllte Raute** befindet
 - Die Raute ist am Ganzen



Ein Student ist Teil einer Lehrveranstaltung

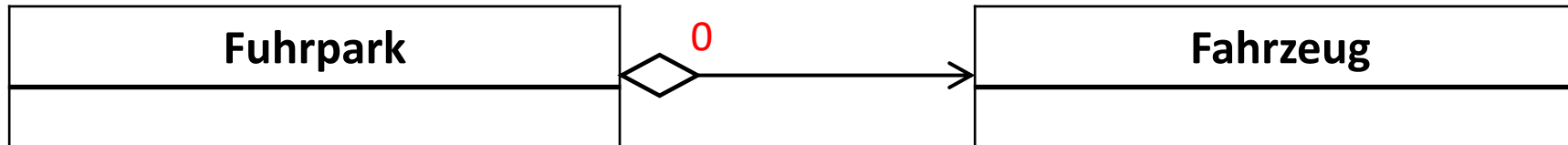
Eine Lehrveranstaltung ist das Ganze – sie besteht optional aus Studenten

Beziehungstypen



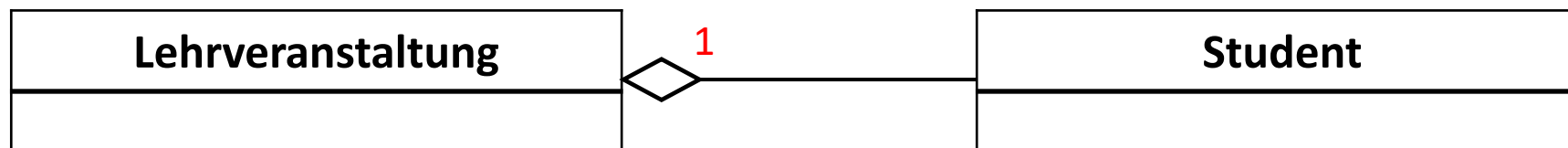
Aggregation – Richtung & Multiplizität

- Die Multiplizität am aggregierenden Ende hängt mit der Richtung der Beziehung zusammen
 - **gerichtet:** Multiplizität = 0. Das Teil kennt das Ganze nicht



*Ein Fahrzeug ist Teil eines Fuhrparks
Das Fahrzeug kennt den Fuhrpark, von dem es Teil ist, nicht*

- **ungerichtet:** Multiplizität = 1. Das Teil kennt das Ganze. Es gehört zu genau einem Ganzen.



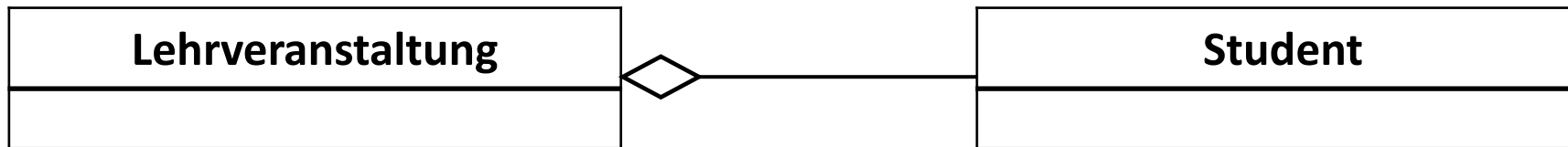
*Ein Student ist Teil einer Lehrveranstaltung
Ein Student kennt die Lehrveranstaltung, von der er Teil ist
Annahme: Ein Student kann zeitgleich nur an einer Lernveranstaltung teilnehmen*

Beziehungstypen



Aggregation – Implementierung

- Eine Aggregation wird meist mit Membervariablen implementiert – wie auch die Assoziation
 - Die Teil-Klasse hat optional eine Membervariable auf das Ganze, wenn die Beziehung ungerichtet ist



```
class Lehrveranstaltung
{
private:
    std::vector<Student*> m_studenten;
};
```

Keine Nullpointer! Muss vorhanden sein!
(Kann aber leer sein = keine Elemente vorhanden)

```
class Student
{
private:
    Lehrveranstaltung* m_lehrveranstaltung;
};
```

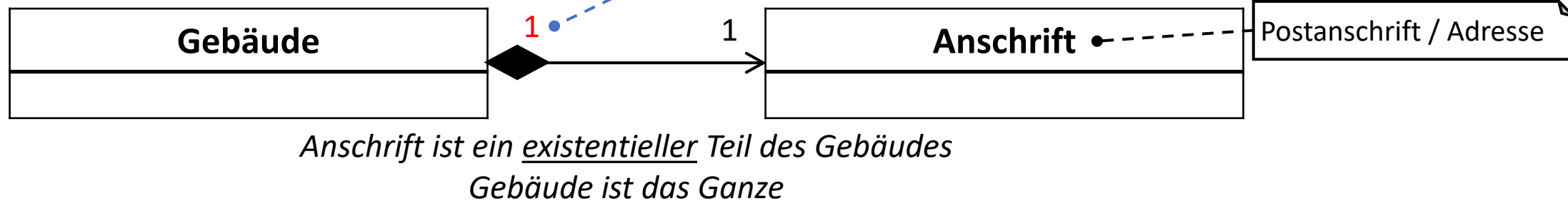
Teil von genau einem Ganzen. Kein Nullpointer!

Beziehungstypen



Komposition

- Eine **Komposition** bei UML-Klassendiagrammen bedeutet, dass eine Klasse nicht ohne eine andere Klasse existieren kann
 - Die Klasse ist ein **existentieller** Teil einer anderen Klasse!
 - Wenn das Ganze gelöscht wird, wird auch der Teil gelöscht
- Kompositionen werden mit einer durchgezogenen Linie dargestellt, an deren Ende sich eine **ausgefüllte Raute** befindet
 - Die Raute ist am Ganzen



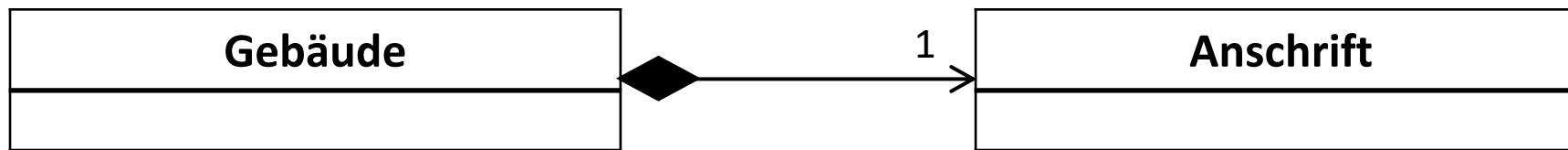
Kompositionen werden in der Praxis häufig vermieden, da sie sehr bindend sind

Beziehungstypen



Komposition – Implementierung

- Eine Komposition ist mindestens gerichtet vom Ganzen zum Teil
 - Kennt auch das Teil das Ganze, ist die Komposition ungerichtet – wie bei der Aggregation
 - Die Multiplizität am komponierenden Ende ist jedoch immer 1
- Kompositionen müssen im Gegensatz zu Aggregationen **bindend** umgesetzt werden, sodass das Teil nur und immer zusammen mit dem Ganzen zerstört wird



```
class Gebäude
{
private:
    Anschrift m_anschrift;
};
```

Implementierung ohne Zeiger oder Referenz sichert Bindung.

- Wird zusammen mit Gebäude-Objekt erzeugt
- Wird zusammen mit Gebäude-Objekt zerstört

Bei Nutzung von Smart-Pointern kann die Variable auch als `std::unique_ptr` umgesetzt werden, um die einmalige Bindung zu sichern

Beziehungstypen

Zusammenfassung



- **Abhängigkeiten** und **Assoziationen** beschreiben die Beziehung zwischen zwei Klassen
 - **Abhängigkeiten** beschreiben eine schwache kurzfristige Beziehung der Klassen
 - **Assoziationen** beschreiben eine starke langfristige Beziehung der Klassen
 - Die **Rolle** und **Multiplizität** von Instanzen einer Klasse wird ans Ende der Assoziationslinie geschrieben
 - Ist eine Abhängigkeit oder Assoziation **gerichtet**, so steht nur eine Klasse in Beziehung zu einer anderen, nicht umgekehrt
 - Assoziationen können direkt als **Membervariablen** einer Klasse umgesetzt werden
- **Aggregationen** und **Kompositionen** beschreiben eine Teil-Ganzes-Beziehung
 - Bei **Aggregationen** können Teile ohne das Ganze existieren
 - Bei **Kompositionen** können Teile nicht ohne das Ganze existieren

Beziehungstypen

Verständnisfragen

PIN: 886080

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Eine Abhängigkeit entsteht, wenn eine Klasse eine andere als Methodenparameter verwendet.
- Hat eine Klasse eine Membervariable einer anderen Klasse, stellt dies eine Abhängigkeitsbeziehung zwischen den Klassen dar.
- Eine Assoziation ist eine langfristige Beziehung zwischen zwei Klassen.
- Setzt sich eine Klasse aus mehreren anderen Klassen zusammen, spricht man von einer Aggregation oder Komposition.
- Eine Aggregation impliziert, dass das Leben des Teils vollständig vom Leben des Ganzen abhängt.
- Eine Komposition kann durch direkte Instanziierung einer Membervariable, ohne Zeiger oder Referenzen, modelliert werden.

■ Welche Beziehung kann durch diese C++-Definition implementiert worden sein?

```
class Material  
{  
    Textur* m_tex;  
}
```

- Abhängigkeit
- Aggregation
- Assoziation
- Komposition



Schnittstellen

Lernziele

- Die Bedeutung von Schnittstellen im Kontext von Klassen wurde verstanden
 - Schnittstellenklassen können aus allgemeinen Definitionen in C++ implementiert werden
 - Der Unterschied zwischen der Erweiterung und der Realisierung von Schnittstellen wurde verstanden
 - Eine Klasse kann auf Basis mehrerer Schnittstellen realisiert werden

Schnittstellen



Schnittstellenklasse (interface)

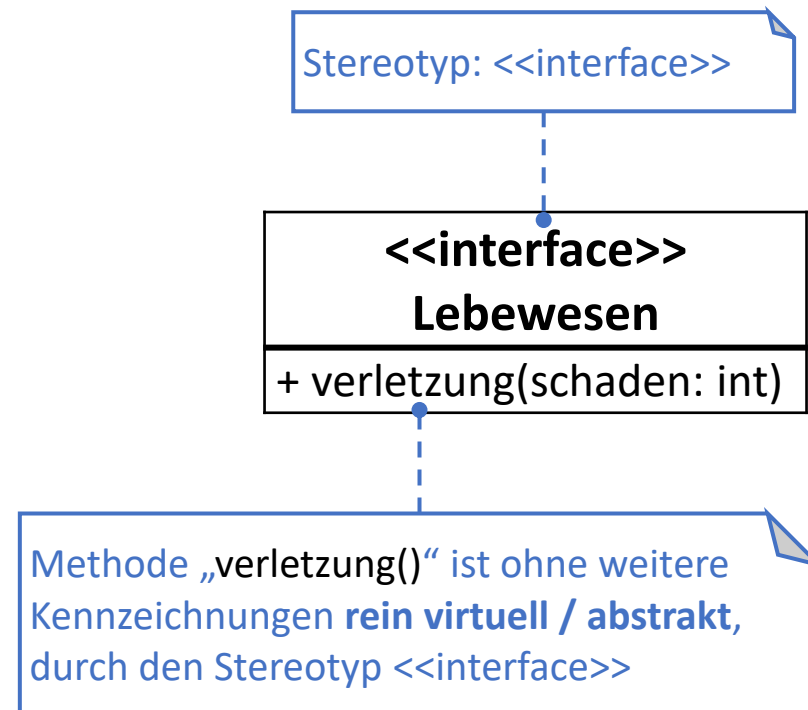
- **Rückblick: Abstrakte Methoden & Klassen**
 - Eine Methode, die nur deklariert, aber nicht definiert wird, ist eine **rein virtuelle Methode** und wird mit **= 0** am Ende der Deklaration gekennzeichnet
 - Klassen, die mindestens eine rein virtuelle Methode besitzen, sind **abstrakt**
- Eine **Schnittstellenklasse** (*interface*) ist eine Klasse, die ausschließlich aus rein virtuellen Methoden besteht
 - Enthalten die Deklarationen aller Operationen, die erbende Klassen implementieren müssen
 - Erzwingen dadurch, dass die deklarierten Funktionalitäten vorhanden sind, ohne eine konkrete Umsetzung anzugeben
 - Besitzen entsprechend für keine Operation eine Definition
 - Besitzen keine Membervariablen

Schnittstellen



Schnittstellenklasse (interface) – UML

- UML-Klassendiagramme kennzeichnen Schnittstellenklassen mit dem *Stereotyp* **<<interface>>**

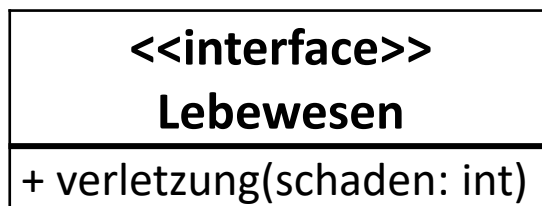


Schnittstellen

Implementierung



- In C++ existiert kein Keyword (oder andere syntaktische Funktionen) für die Definition von Schnittstellenklassen
 - Schnittstellenklassen müssen als „gewöhnliche“ Klassen definiert werden, bei denen...
 - **jede Methode rein virtuell** deklariert ist (*virtual ... = 0*) und...
 - **keine Membervariablen** vorhanden sind
 - Außerdem muss in C++ darauf geachtet werden, dass zusätzlich zu den Schnittstellenfunktionen immer ein **virtueller Destruktor** definiert wird!
 - Sonst werden Destruktoren von Kindklassen (= Realisierungen der Schnittstelle) nicht korrekt aufgerufen (siehe Basic Objektorientierung)



UML-Schnittstellenklasse

```
class Lebewesen
{
public:
    virtual ~Lebewesen() = default;

    virtual void verletzung(int schaden) = 0;
};
```

C++ Implementierung der Schnittstellenklasse

Virtueller Destruktor
Kann bei korrekter
Implementierung immer
default gesetzt werden

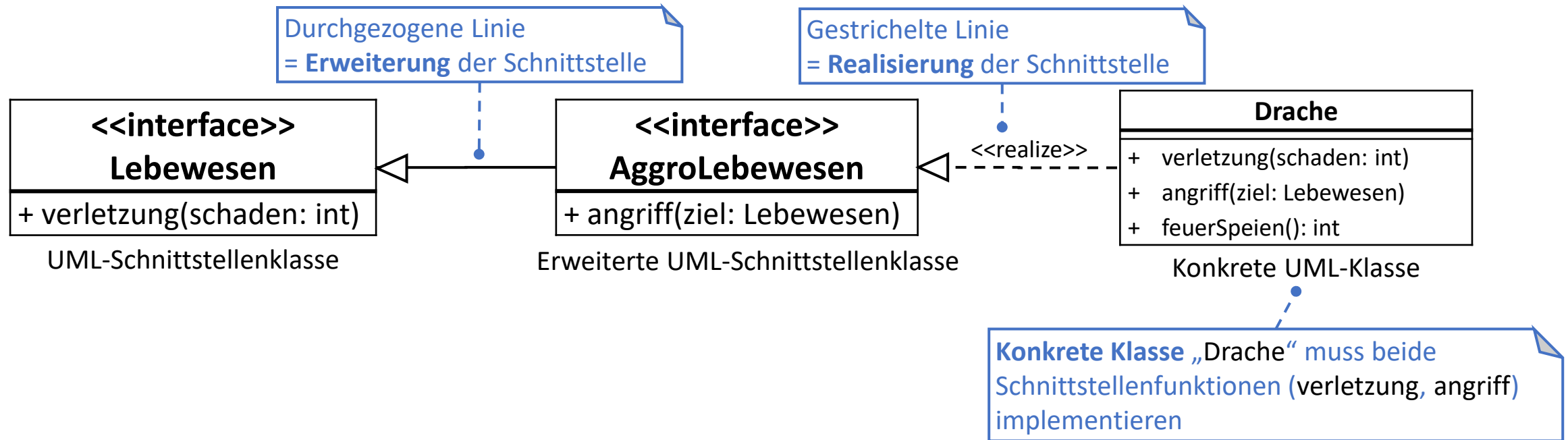
Schnittstellenfunktion

Schnittstellen

Erweiterung & Realisierung



- In UML-Klassendiagrammen wird unterschieden zwischen der **Erweiterung** einer Schnittstellenklasse und einer **Realisierung** der Schnittstellenklasse
 - Erweiterungen werden wie die Vererbung anderer Klassen dargestellt
 - Realisierungen werden mit einer gestrichelten Linie dargestellt, optional mit der Property `<<realize>>`



Schnittstellen

Erweiterung & Realisierung



- Bei der Umsetzung von Erweiterungen und Implementierungen von Schnittstellenklassen in C++ gibt es keinen syntaktischen Unterschied
 - Bei der Erweiterung wird von der Schnittstellenklasse geerbt
 - Bei der Realisierung wird ebenfalls von der Schnittstellenklasse geerbt
 - Zusätzlich werden die Schnittstellenfunktionen implementiert



```
class Lebewesen
{
public:
    virtual ~Lebewesen() = default;
    virtual void verletzung(int schaden) = 0;
};
```

```
class AggroLebewesen : public Lebewesen
{
public:
    virtual void angriff(Lebewesen* ziel) = 0;
};
```

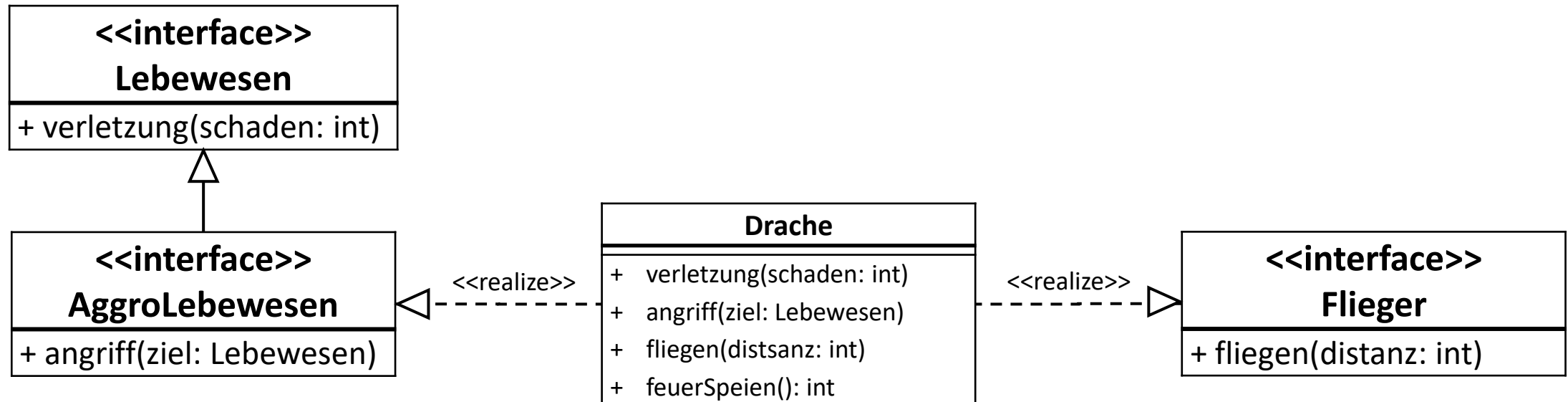
```
class Drache : public AggroLebewesen
{
public:
    void verletzung(int schaden) override { ... }
    void angriff(Lebewesen* ziel) override { ... }
    int feuerSpeien() { ... }
};
```

Schnittstellen

Mehrfachvererbung



- Bei der Realisierung von Schnittstellen kommt es häufig vor, dass eine Klasse mehrere Schnittstellen implementieren muss
- Beispiel: „Drache“ soll zusätzlich die Schnittstelle „Flieger“ realisieren



Schnittstellen

Mehrfachvererbung



- Eine Klasse kann von mehreren Klassen erben – dies wird als **Mehrfachvererbung** (*multiple inheritance*) bezeichnet
 - Die Elternklassen werden nach dem Doppelpunkt mit **einfachen Kommas** (,) getrennt aufgelistet
 - Für jede Elternklasse muss die Zugriffs-Spezifikation (private, protected, public) explizit angegeben werden; ohne Angabe wird private geerbt

```
class Drache : public AggroLebewesen, public Flieger
{
public:
    void verletzung(int schaden) override { ... }
    void angriff(Lebewesen* ziel) override { ... }
    void fliegen(int distanz) override { ... }
    int feuerSpeien() { ... }
};
```

Klasse „Drache“ erbt von zwei Elternklassen

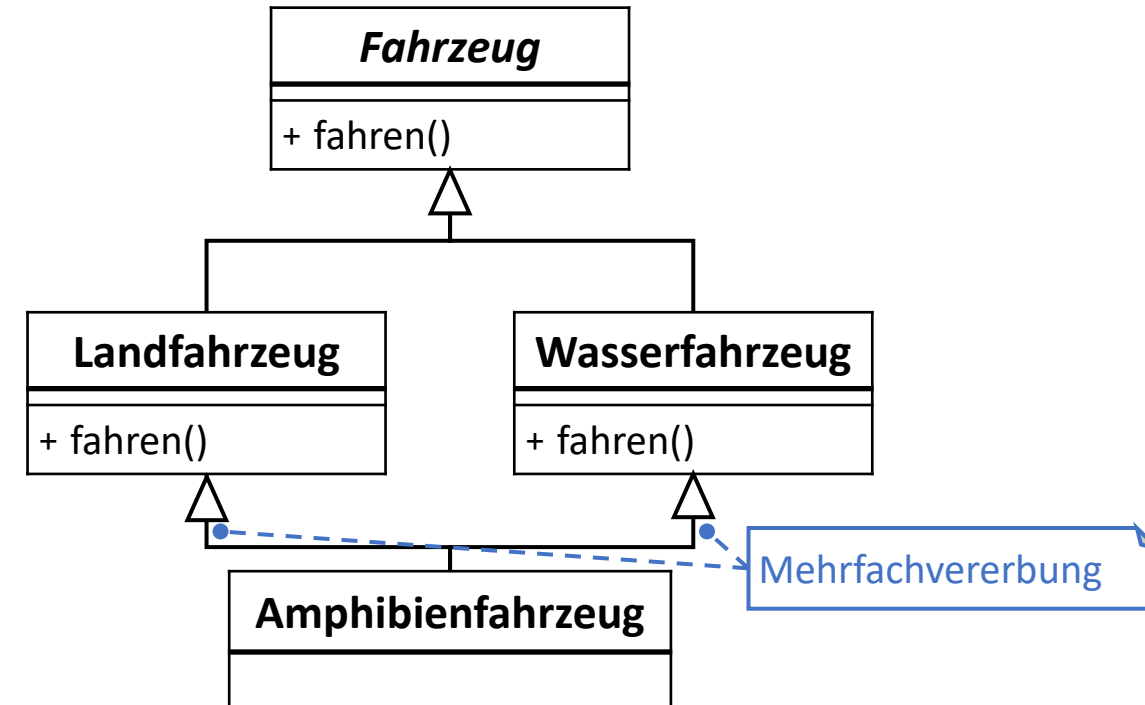
Schnittstellen

Mehrfachvererbung

- Durch Mehrfachvererbung können Probleme entstehen
 - **Diamantproblem:** Erbt eine Klasse von zwei Elternklassen, die wiederum von der gleichen Elternklasse abstammen, wird der Zugriff auf Member der Basisklasse mehrdeutig
 - **Andere Mehrdeutigkeiten:** Heißt ein Member der beiden Elternklassen gleich, kommt es zu Mehrdeutigkeiten in der Kindklasse
 - **Konstruktor & Destruktor Reihenfolge:** Die Reihenfolge der Erzeugung und Zerstörung der Elternklassen wird undurchsichtig



■ Beispiel: Diamantproblem



```
void main()
{
    Amphibienfahrzeug amphi;
    amphi.fahren();
}
```

Welche Methode soll hier aufgerufen werden?



→ Mehrfachvererbungen sollten außerhalb von Schnittstellenklassen vermieden werden



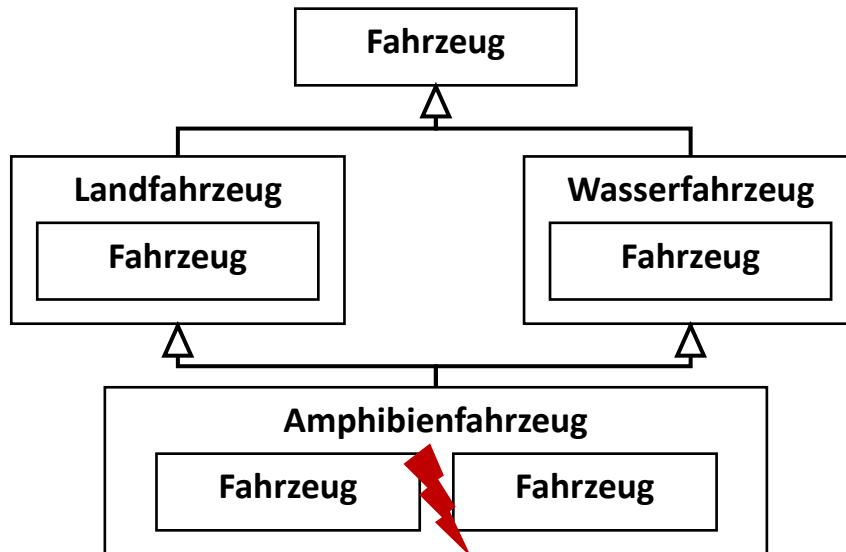
Schnittstellen



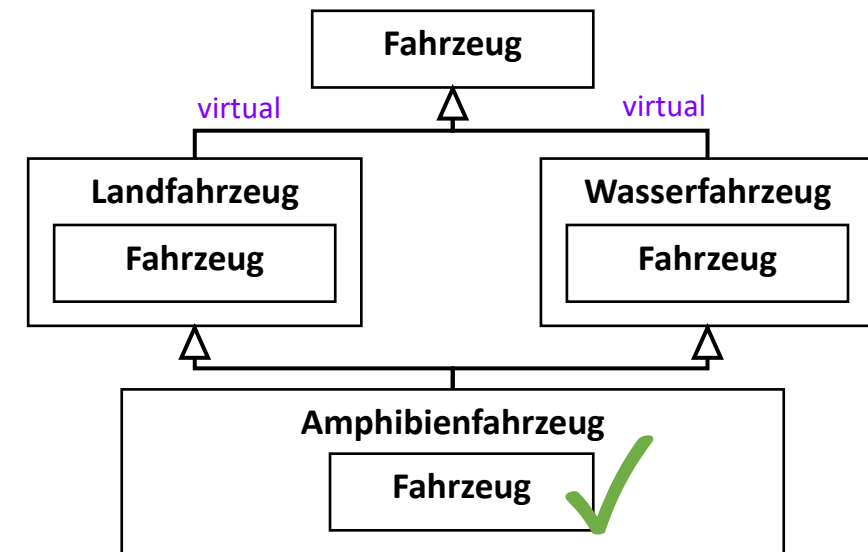
Mehrfachvererbung – Virtuell

- Kann eine Mehrfachvererbung von der gleichen Elternklasse nicht vermieden werden, können mit Hilfe von **virtueller Vererbung** Mehrdeutigkeiten verhindert werden
 - Dabei wird in die letzte Kindklasse nur eine Instanz der Elternklasse kompiliert

```
class Fahrzeug { ... };  
class Landfahrzeug : public Fahrzeug { ... };  
class Wasserfahrzeug : public Fahrzeug { ... };  
class Amphibienfahrzeug : public Landfahrzeug, public Wasserfahrzeug { ... };
```



```
class Fahrzeug { ... };  
class Landfahrzeug : virtual public Fahrzeug { ... };  
class Wasserfahrzeug : virtual public Fahrzeug { ... };  
class Amphibienfahrzeug : public Landfahrzeug, public Wasserfahrzeug { ... };
```



Schnittstellen

Zusammenfassung



- **Schnittstellenklassen** (*interfaces*) definieren Methoden, die *realisierende* Klassen implementieren müssen
 - Schnittstellen definieren niemals Membervariablen oder Methodenimplementierungen
- **Schnittstellenklassen** können *erweitert* und *realisiert* werden
 - Bei der **Erweiterung** entsteht eine weitere Schnittstellenklasse
 - Bei der **Realisierung** entsteht eine konkrete Klasse, die alle abstrakten Methoden einer Schnittstellenklasse implementiert
- Durch **Mehrfachvererbung** (*multiple inheritance*) kann eine Klasse mehrere Schnittstellen realisieren
 - Mehrfachvererbung sollte nur für virtuelle Basisklassen (z.B. Schnittstellenklassen) angewendet werden, da sonst große Probleme mit Mehrdeutigkeiten entstehen können

Schnittstellen

Verständnisfragen

PIN: 886080

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Schnittstellen werden in C++ mit dem Keyword **interface** definiert.
- Eine Schnittstellenklasse besteht aus rein virtuellen Methoden.
- Eine Schnittstellenklasse ist ein Synonym für eine abstrakte Klasse.
- Schnittstellenklassen sollten neben den rein virtuellen Methoden einen virtuellen Destruktor definieren.
- Wird eine Schnittstelle realisiert, so werden die Methoden einer Schnittstelle implementiert.
- Eine Schnittstelle wird in C++ mit dem Keyword **realize** realisiert.
- Eine Schnittstelle ist realisiert, wenn mindestens eine der virtuellen Methoden der Schnittstelle implementiert wurde.
- Eine konkrete Klasse kann maximal eine Schnittstelle realisieren.



Code-Organisation

Lernziele

- Projekte können für komplexe Beziehungen zwischen Klassen effizient organisiert werden
 - Probleme mit Abhängigkeiten zwischen Klassen und Header-Dateien wurden verstanden
 - Das Prinzip der **Vorausdeklaration** kann angewendet werden
 - Header-Dateien können in **vorkompilierten Headern** zusammengefasst werden
 - Klassen und andere Definitionen können **Namensräumen** untergeordnet werden

Code-Organisation



Vorausdeklarationen

- Als **Vorausdeklaration** (*forward declaration*) bezeichnet man die Deklaration eines Typs (Klasse, Struct, Funktion, Variable, ...), **bevor** dieser das erste Mal **definiert** wird
- Das Vorausdeklarieren ist nötig, wenn die volle Typdefinition noch nicht verfügbar ist, aber bereits benötigt wird
- Häufig kommen Vorausdeklarationen zum Einsatz bei:
 - Auflösung **zyklischer Abhängigkeiten**
 - Definitionen von Funktions**prototypen**
 - Reduzierung von **Kompilierzeiten**
 - Unabhängigkeit bei **Header-Inkludierung**

Code-Organisation



Vorausdeklarationen

- Bei einer Vorausdeklaration wird der Name und Typ eines Objekts angegeben, ohne jegliche Details (wie Member, Funktionsblöcke, ...)

```
class Drache;
```

Vorausdeklaration der **Klasse** „Drache“

```
enum Color;
```

Vorausdeklaration der **Enumeration** „Color“

```
struct ArrayWithSize;
```

Vorausdeklaration des **Structs** „ArrayWithSize“

- Durch eine Vorausdeklaration weiß der Compiler bereits, um welche Art es sich bei einem Typnamen handelt
 - „'Drache' ist eine Klasse“, „'Color' ist eine Enumeration“, ...
 - Die vollständige Definition des Typs folgt zu einem späteren Zeitpunkt
 - Dies erlaubt es Zeiger und Referenzen auf den Typ bereits zu nutzen, bevor dieser vollständig definiert ist
 - Einzelne Details vorausdeklarerter Typen können jedoch noch nicht abgefragt werden (Variablen, Funktionen, ...)

```
class Drache; // Vorausdeklaration

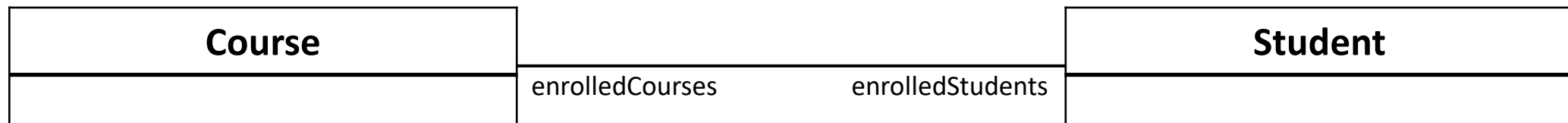
void angreifen(Drache& drache) // OK: Referenz
{
    drache.angreifen(); // ERROR:
                        // Zugriff auf Detail
}
```

Code-Organisation



Vorausdeklarationen – zyklische Abhängigkeit

- Zyklische Abhängigkeiten treten häufig beim Entwurf von Klassen auf; Besonders bei bidirektionalen (= ungerichteten) Beziehungen
- Beispiel: Beziehung zwischen einer Lehrveranstaltung (course) und einem Studenten (student)



Einer Lehrveranstaltung gehören eingeschriebene (enrolled) Studenten an und ein Student ist in Lehrveranstaltungen eingeschrieben

→ Zyklische Abhängigkeit

Code-Organisation



Vorausdeklarationen – zyklische Abhängigkeit

- Bei der Ausformulierung von Klassen mit zyklischen Abhängigkeiten kommt es unweigerlich zu einem **Problem der Definitionsreihenfolge**

```
class Course
{
private:
    Student* m_enrolledStudents[...];
};

class Student
{
private:
    Course* m_enrolledCourses[...];
};
```

Klasse Student ist
noch nicht bekannt,
da sie erst darunter
definiert wird

```
class Student
{
private:
    Course* m_enrolledCourses[...];
};

class Course
{
private:
    Student* m_enrolledStudents[...];
};
```

Klasse Course ist
noch nicht bekannt,
da sie erst darunter
definiert wird

➔ Das Problem lässt sich nicht durch Vertauschen der Definitionsreihenfolge lösen

Code-Organisation



Vorausdeklarationen – zyklische Abhängigkeit

- In diesen Fällen muss eine der beiden Klassen **vorausdeklariert** werden, damit es zu keinem Compilerfehler kommt

```
class Student;
```

Vorausdeklaration der Klasse `Student`

```
class Course
```

```
{
```

```
private:
```

```
    Student* m_enrolledStudents[...];
```

```
};
```

✓ Die Klasse `Student` kann nun verwendet werden bevor sie definiert wurde

```
class Student
```

```
{
```

```
private:
```

```
    Course* m_enrolledCourses[...];
```

```
};
```

Anschließend wird die Klasse `Student` (wie gehabt) definiert

Code-Organisation



Vorausdeklarationen – Funktionsprototypen

- Die Vorausdeklaration von Funktionen bezeichnet man als **Funktionsprototypen**
 - Dabei wird nur die **Signatur** (Name, Parameter, Rückgabewert, ...) deklariert
- Beispiel:

```
float square(float x);
```

Funktionsprototyp von square(x)

```
int main()
```

```
{  
  std::cout << "Quadrat von 4 ist: " << square(4);  
}
```

Die Funktion square(x) kann verwendet werden bevor sie definiert wurde

```
float square(float x)
```

```
{  
  return x * x;  
}
```

Die Definition der Funktion square(x) erfolgt zu einem späteren Zeitpunkt

```
Quadrat von 4 ist: 16
```

Code-Organisation



Vorausdeklarationen – Kompilierzeit & Header

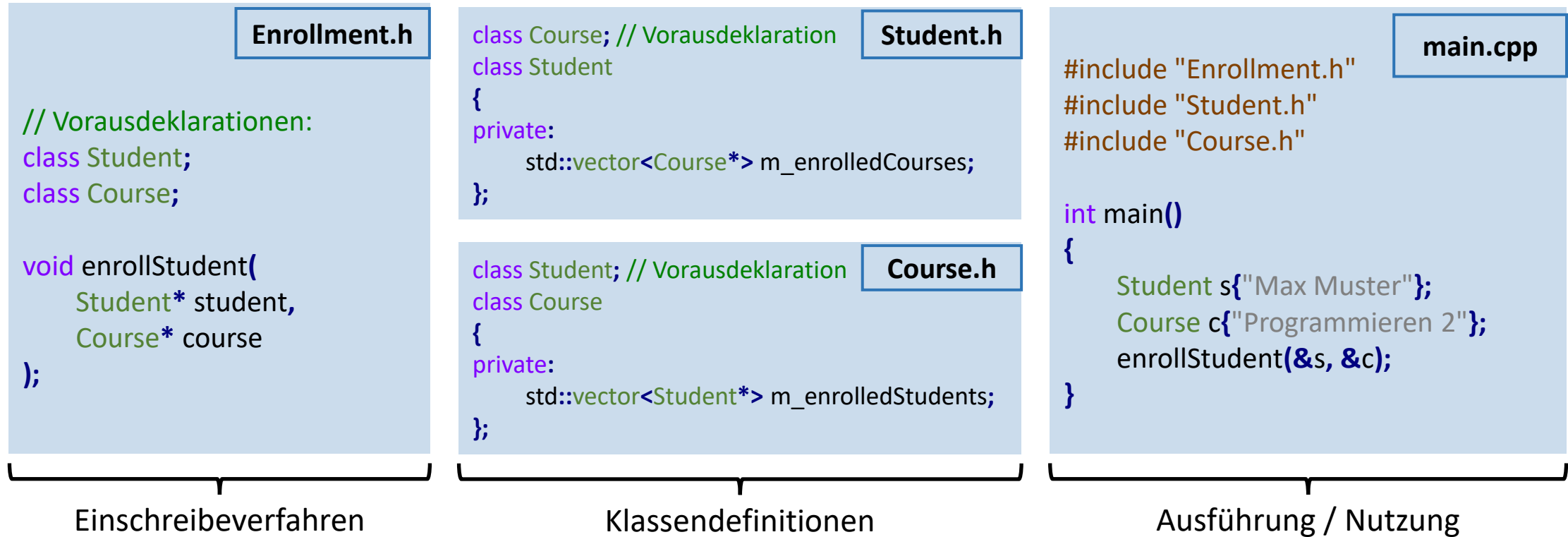
- Das Inkludieren von **Headern in Headern** ist häufig mit einem deutlichen **Anstieg der Kompilierzeiten** verbunden
 - Da Header von jeder Compiler-Einheit (.cpp / Source) eingebunden werden
- Besonders in Header-Dateien kann es deshalb sinnvoll sein, größere Definitionen vor auszudeklariieren und damit Kompilierzeit zu sparen
- Außerdem wird so eine **Unabhängigkeit** hergestellt, welcher Header zuerst inkludiert werden muss

Code-Organisation



Vorausdeklarationen – Kompilierzeit & Header

- Beispiel: Einschreibung (enrollment) von Studenten in Lehrveranstaltungen



Code-Organisation

Vorausdeklarationen – Kompilierzeit & Header



Enrollment.h

```
// Vorausdeklarationen:  
class Student;  
class Course;  
  
void enrollStudent(  
    Student* student,  
    Course* course  
);
```

Durch die Vorausdeklaration der Klassen `Student` und `Course` werden diese nicht bei der Inkludierung von `Enrollment.h` doppelt geparkt.

main.cpp

```
#include "Enrollment.h"  
#include "Student.h"  
#include "Course.h"  
  
int main()  
{  
    Student s{"Max Muster"};  
    Course c{"Programmieren 2"};  
    enrollStudent(&s, &c);  
}
```

Code-Organisation

Vorausdeklarationen – Kompilierzeit & Header



Auch kann die Reihenfolge der Inkludierung von `Enrollment.h`, `Student.h` und `Course.h` beliebig vertauscht werden, ohne dass undefinierte Typen entstehen
(Da jeder Header seine benötigten Typen vorausdeklariert hat)

```
#include "Enrollment.h"
#include "Student.h"
#include "Course.h"

int main()
{
    Student s{"Max Muster"};
    Course c{"Programmieren 2"};
    enrollStudent(&s, &c);
}
```

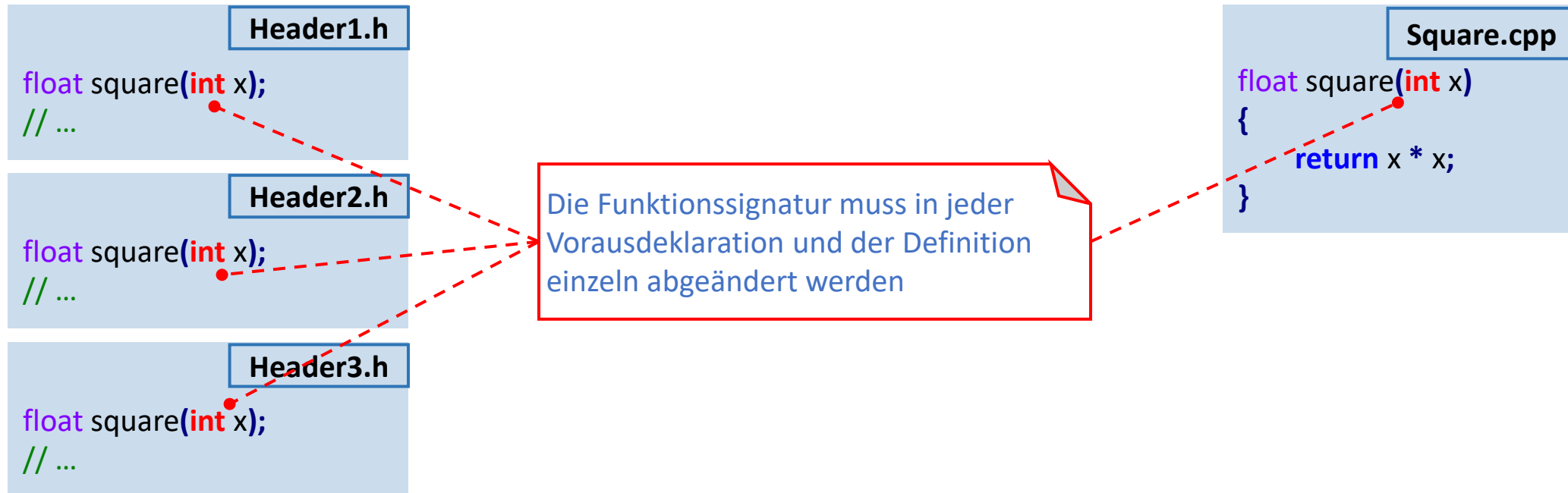
main.cpp

Code-Organisation



Vorausdeklarationen – Probleme

- Trotz ihrer Vorteile sollten Vorausdeklarationen sparsam eingesetzt werden, da der **Wartungsaufwand** durch ihren Einsatz steigt
- Beispiel: Der Parameter einer vorausdeklarierten Funktion wird von **int** zu **float** geändert



Code-Organisation: Vorausdeklarationen



Zusammenfassung

- **Vorausdeklarationen** erlauben es, einen Typen (Klasse, Struct, Funktion, Variable, ...) anzugeben, bevor dieser definiert wird
- Vorausdeklarationen können **zyklische Abhängigkeiten** in Headern **auflösen**, sowie **Kompilierzeiten reduzieren**
- Vorausdeklarationen **steigern den Wartungsaufwand**, da Änderungen an Typen in allen Vorausdeklarationen redundant vorgenommen werden müssen

Code-Organisation



Namenskollisionen

- Bei komplexen Projekten besteht die Gefahr, dass **Typnamen** (Klassen, Structs, Funktionen, ...) mehrfach vorkommen
 - Bei deren Einbindung kommt es dann zu einer **Mehrdeutigkeit** (*ambiguity*), da ein Name unterschiedlich interpretiert werden kann
 - Dabei spricht man von einer **Namenskollision** (*name collision*)
 - Dies führt fast immer zu einem Compilerfehler und zwingt den Programmierer dazu, eigens-definierte Typnamen zu ändern
- Besonders häufig kommt es bei der Einbindung von (Fremd-)Bibliotheken zu Namenskollisionen

Code-Organisation



Namenskollisionen – Beispiel

- Beispiel: Für eine Game-Engine wird eine Physik- und eine Grafik-API eingebunden, die beide eine Konstante „PI“ einführen

```
PhysicsApi.h

const float PI { 3.14159265358f };

class RigidBody
{
    void applyForce(float angle, float strength);
    ...
};
```

Bibliothek 1

```
GraphicsApi.h

const float PI { 3.14159265358f };

class Transform
{
    void rotate(float angle);
    ...
};
```

Bibliothek 2

```
main.cpp

#include "PhysicsApi.h"
#include "GraphicsApi.h"

int main()
{
    RigidBody rb;
    Transform t;
    ...
}
```

Nutzendes Programm

Bei der Einbindung beider Bibliotheksheader kommt es zur Namenskollision von PI
→ Die Bibliotheken können nicht zusammen verwendet werden

Code-Organisation



Namensräume

- Um dem Problem von Namenskollisionen zuvorzukommen, können Definitionen **Namensräumen** (*namespaces*) untergeordnet werden
- Namensräume werden über einen **Geltungsbereich** (*scope*) aufgespannt

Header

```
namespace MyApplication
{
    class Texture
    {
    public:
        Texture();
    };
}
```

Source

```
namespace MyApplication
{
    Texture::Texture()
    {
        // ...
    }
}
```

Alternativ kann der Namensraum nach Deklaration auch *zeilenbasiert* (ohne Scope) verwendet werden

Source

```
MyApplication::Texture::Texture()
{
    // ...
}
```

Code-Organisation



Namensräume – Einbindung

- Innerhalb des Geltungsbereichs eines Namensraums sind alle Definitionen des eigenen Namensraums sichtbar und müssen nicht explizit eingebunden werden

Header

```
namespace MyApplication
{
    class Texture
    {
    public:
        Texture();
    };

    struct PixelData { /*...*/ }
}
```

Source

```
namespace MyApplication
{
    Texture::Texture()
    {
        PixelData data;
    }
}
```

Zugriff auf „PixelData“ ist ohne Angabe des Namensraums „MyApplication“ innerhalb von diesem möglich

Code-Organisation



Namensräume – Einbindung – Zeilenbasiert

- Außerhalb des Geltungsbereichs eines Namensraums, muss dieser zuerst **explizit angegeben** werden, bevor auf den jeweiligen Typ zugegriffen werden kann
 - Zeilenbasiert erfolgt das über die Angabe des Namensraums gefolgt von zwei Doppelpunkten (::) – wie bei Klassen

Header

```
namespace MyApplication
{
    class Texture
    {
    public:
        Texture();
    };
}
```

Source

```
int main()
{
    MyApplication::Texture texture;
}
```

Zeilenbasierte Einbindung des Namensraums „MyApplication“ um auf den Typen „Texture“ zuzugreifen

Code-Organisation



Namensräume – Einbindung – Bereichsbasiert

- Mit der „**using namespace**“ Anweisung kann ein Namensraum in den aktuellen Geltungsbereich (scope) eingebunden werden

Header

```
namespace MyApplication
{
    class Texture
    {
    public:
        Texture();
    };
}
```

Source

```
int main()
{
    using namespace MyApplication;
    Texture texture;
}
```

Bereichsbasierte Einbindung des Namensraums „**MyApplication**“ um auf den Typen „**Texture**“ zuzugreifen

Definitionen des Namensraums „**MyApplication**“ sind bis zum Ende des Scopes sichtbar

Code-Organisation



Namensräume – Einbindung – Beispiel

- Beispiel: Die Physik- und Grafik-API aus vorherigem Beispiel definieren ihre Typen nun jeweils in einem eigenen Namensraum

```
namespace Physics
{
    const float PI { 3.14159265358f };

    class RigidBody
    {
        void applyForce(float angle, float strength);
        ...
    };
}
```

PhysicsApi.h

```
namespace Graphics
{
    const float PI { 3.14159265358f };

    class Transform
    {
        void rotate(float angle);
        ...
    };
}
```

GraphicsApi.h

```
#include "PhysicsApi.h"
#include "GraphicsApi.h"

int main()
{
    Physics::RigidBody rb;
    Graphics::Transform t;
    ...
}
```

main.cpp

✓ keine Namenskollision

Bei der Einbindung beider Bibliotheksheader kommt es zu keinen Namenskollisionen mehr

→ Die Bibliotheken können zusammen verwendet werden

Code-Organisation



Namensräume – Verschachtelung

- Mehrere Namensräume können ineinander verschachtelt werden:

Header

```
namespace MyApplication
{
    namespace Graphics
    {
        class Texture
        {
        public:
            Texture();
        };
    }
}
```



Header

```
namespace MyApplication::Graphics
{
    class Texture
    {
    public:
        Texture();
    };
}
```

Verkettete Definition von zwei Namensräumen

→ Die Klasse „Texture“ befindet sich nun im Namensraum „Graphics“ der unter „MyApplication“ liegt

Code-Organisation



Namensräume – Globaler Namensraum

- Alle Definitionen, die nicht explizit einem Namensraum untergeordnet sind, befinden sich im **globalen Namensraum** (*global namespace*)
 - Das sind insbesondere Definitionen von Bibliotheken aus der Programmiersprache **C**
 - Deshalb enthalten C-Bibliotheken häufig **Präfixe** auf Typnamen (z.B. **vk**CreateDevice (...) bei der „Vulkan“ API, **gl**Begin (...) bei der „OpenGL“ API, ...)
- Ein Grundsatz bei der Programmierung in C++ ist, den globalen Namensraum so wenig wie möglich zu „verschmutzen“
 - Z.B. mit eigenen Klassen, Funktionen, ... außerhalb eines explizit definierten Namensraums
- Der globale Namensraum ist in jedem Kontext ohne explizite Einbindung sichtbar
 - Ein zeilenbasierter Zugriff auf den globalen Namensraum ist dennoch möglich durch einen doppelten Doppelpunkt (::) ohne vorangegangenen Namen:

Zugriff auf den globalen Namensraum (mit ::)
in dem die C-Funktion „puts(...)“ liegt

```
int main()
{
    ::puts("Hallo!");
}
```


Code-Organisation: Namensräume



Zusammenfassung

- Um **Namenskollisionen** vorzubeugen, sollten Typnamen in eigenen Namensräumen definiert werden
 - Im globalen Namensraum sollten so wenig Definitionen wie möglich erfolgen
- Im aktuellen **Kontext** (Funktionsbody, Klassendefinition, ...) sind alle Typen des eigenen und des globalen Namensraums sichtbar
 - Typen anderer Namensräume müssen explizit eingebunden werden
- Namensräume können entweder **zeilenbasiert** (pro Zeile) oder für einen ganzen **Geltungsbereich** (scope) eingebunden werden
- Namensräume können verschachtelt werden
 - Seit **C++17** auch in einer einzigen Zeile

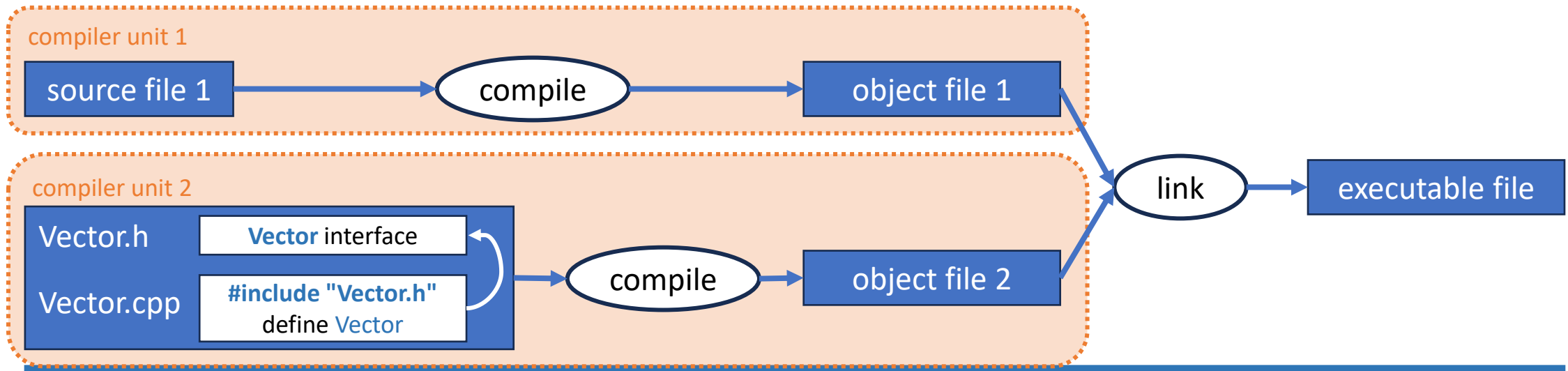
Code-Organisation

Programmstruktur



■ Rückblick: C++ ist eine **Compiler-Sprache**

- Die Quelldateien (.cpp, .h, ...) werden von einem Compiler verarbeitet
- Jede .cpp-Quelldatei wird in einer **Compiler-Einheit** zu einer Objektdatei (.obj) kompiliert, die die Objekte der .cpp-Datei enthält (Typen, Variablen, Maschinencode, Metainformationen, ...)
- Die Objektdateien werden vom Linker zu einem ausführbaren Programm vereint

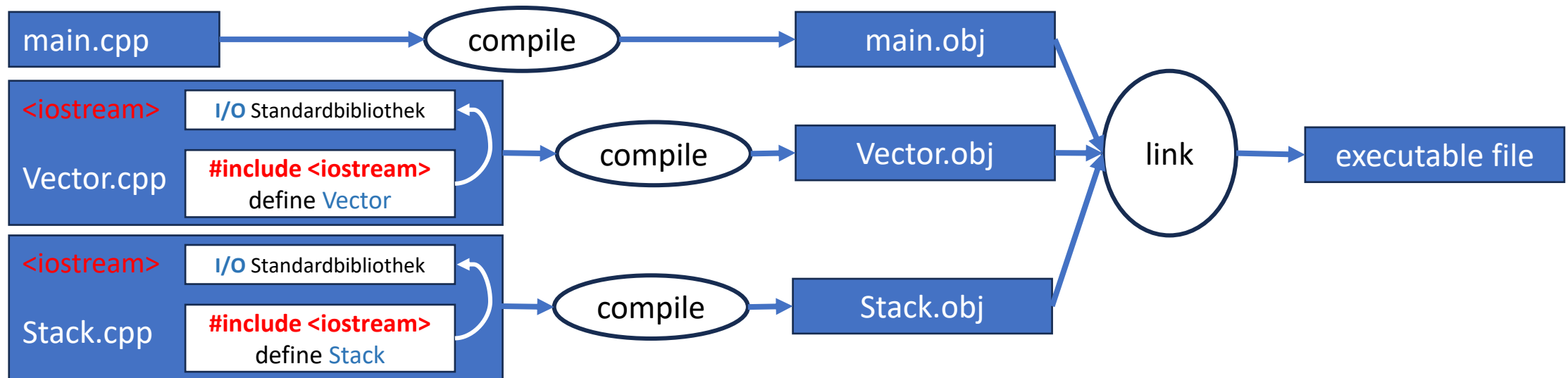


Code-Organisation



Programmstruktur – Header-Dateien

- **Header-Dateien** (.h) werden nicht direkt kompiliert – sie erweitern eine Compiler-Einheit, wenn sie von einer Quelldatei (.cpp) inkludiert werden
 - Dabei werden sie nur im Kontext der inkludierenden .cpp-Datei kompiliert
- Inkludieren mehrere Quelldateien die gleiche Header-Datei, so wird diese mehrfach erneut kompiliert

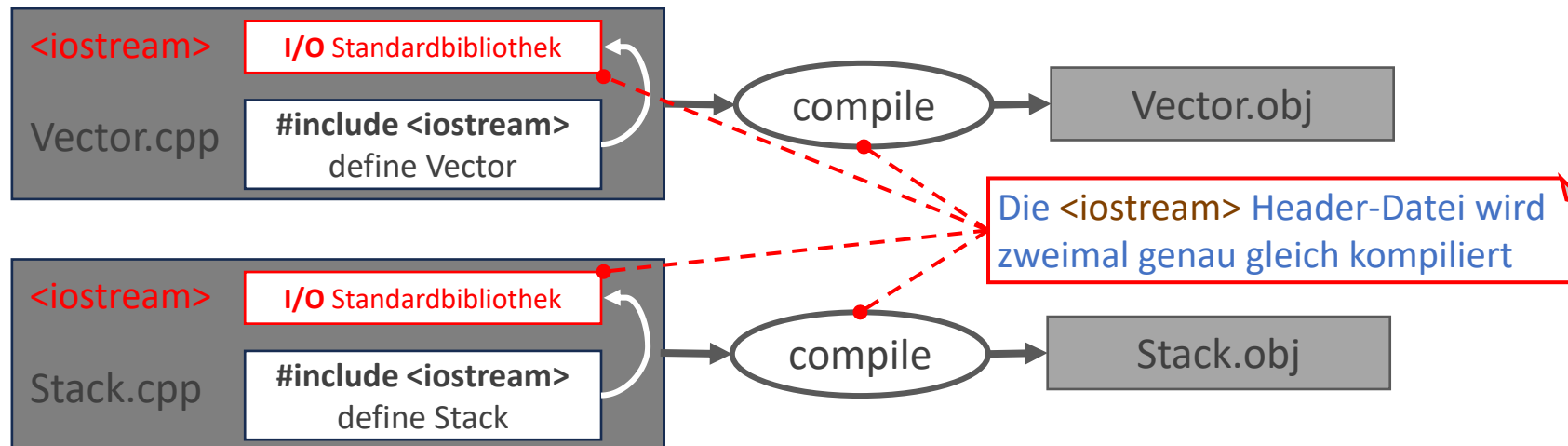


Code-Organisation



Programmstruktur – Header-Dateien

- Auch wenn sich am Inhalt und Kontext einer Header-Datei nichts ändert, wird diese mehrfach, genau gleich, kompiliert



- Das erhöht die Kompilierzeit immens – gerade bei größeren Projekten
 - Z.B. mehrere 100 Quelldateien binden die Standardbibliothek ein (`<iostream>`, `<string>`, `<memory>`, ...)

Code-Organisation



Vorkompilierte Header

- Um die Kompilierzeiten bei Headern zu reduzieren, die häufig inkludiert werden, führten Compiler die **vorkompilierten Header** (*precompiled header*) ein
- Ein vorkompilierter Header ist eine normale Header-Datei, die ...
 - hauptsächlich **andere Header-Dateien inkludiert** (die sonst verstreut in einzelnen .cpp-Dateien inkludiert werden würden),
 - in jeder .cpp-Datei **zu Beginn inkludiert** wird und ...
 - sonst keine Typen (Klassen, Funktionen, ...) selbst definiert (kein Muss, aber gute Vorgehensweise)
- Vorkompilierte Header-Dateien heißen häufig
 - "stdafx.h" – Standard Application Framework – MSVC traditionell
 - "pch.h" – Precompiled Header – GNU, CLang, MSVC modern

Achtung: Vorkompilierte Header sind kein Teil der Sprache C++ und Compiler-abhängig

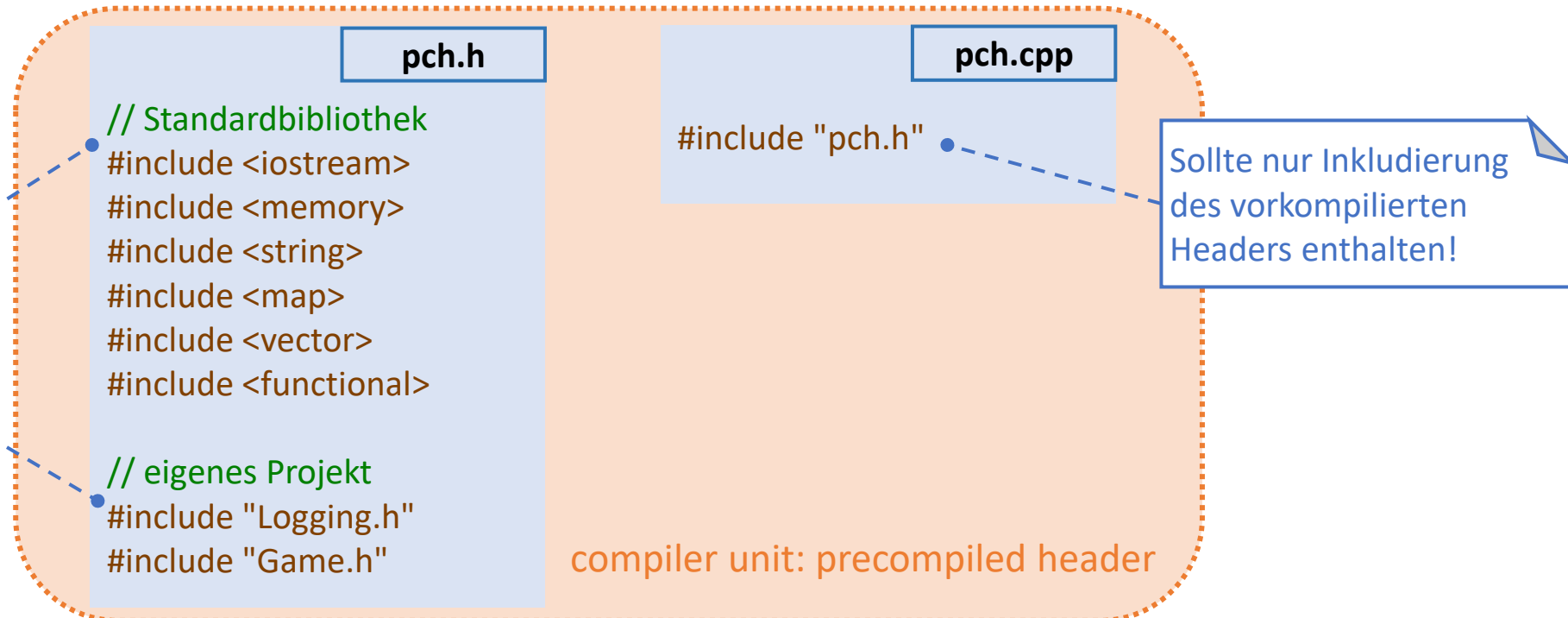
Code-Organisation



Vorkompilierte Header – Kompiliervorgang

- Da vorkompilierte Header auch nur Header-Dateien sind, müssen diese von einer **eigenen Compiler-Einheit** eingebunden werden, um kompiliert zu werden
 - Die Quelldatei des vorkompilierten Headers heißt meist gleich (pch.cpp, stdafx.cpp) und sollte keinen anderen Programmcode enthalten, außer der Inkludierung des Headers

- Beispiel:

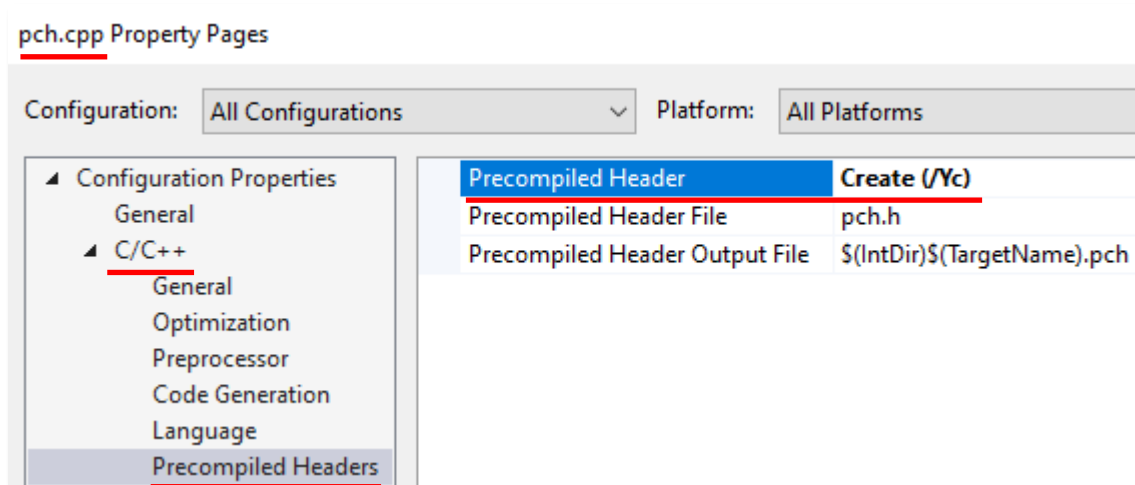


Code-Organisation



Vorkompilierte Header – Kompiliervorgang

- Bei der Quelldatei (pch.cpp) muss dem Compiler vermittelt werden, dass es sich um einen vorkompilierten Header handelt
 - Diese Einstellung ist Compiler-abhängig
- Beispiel: MSVC



Projekteinstellung in Visual Studio auf der **pch.cpp** Datei

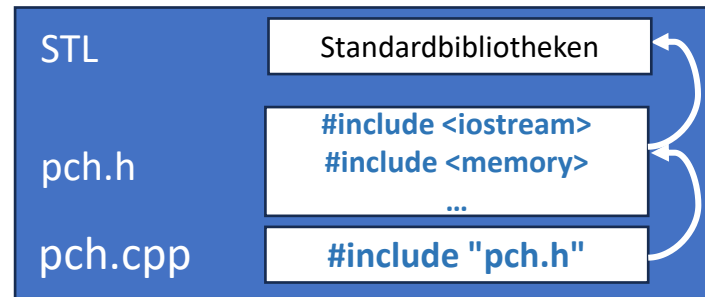
Code-Organisation



Vorkompilierte Header – Kompiliervorgang

- Die Compiler-Einheit des vorkompilierten Headers wird dann vom Compiler **priorisiert** vor allen anderen Compiler-Einheiten übersetzt (**Vorkompilierung**)
 - Die resultierende Objektdatei hat ein eigenes Format (.pch) und steht zu Beginn des Kompilierungsprozesses für andere Compiler-Einheiten bereit

Vorkompilierung:

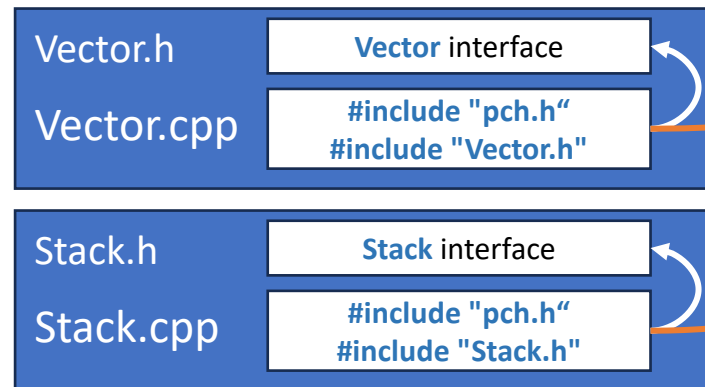


compile

Projekt.pch

Spezielles Format, das Binärdaten der gepackten Header enthält

Hauptkompilierung:



compile

Vector.obj

compile

Stack.obj

Inkludierung des vorkompilierten Headers

link

executable file

Code-Organisation



Vorkompilierte Header – Kompiliervorgang

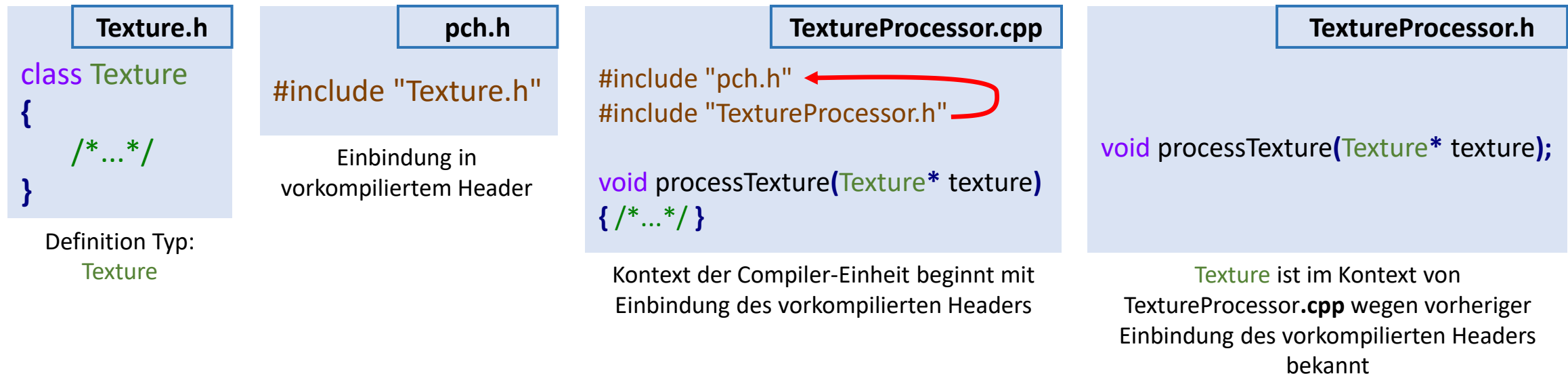
- Durch den vorkompilierten Header werden in diesem inkludierte Header nur noch **einmal kompiliert**, statt jeweils für jede einbindende Compiler-Einheit
→ Erhebliche Reduktion der Kompilierzeit

Code-Organisation



Vorkompilierte Header – Andere Header

- Werden bei einem Projekt vorkompilierte Header benutzt, so muss jede Compiler-Einheit (.cpp-Datei) den **vorkompilierten Header zu Beginn inkludieren**
 - Das führt dazu, dass von Anfang des Kontexts einer .cpp-Datei an **alle Typen** (Klassen, Funktionen, ...) **des vorkompilierten Headers bekannt** sind
 - Daraus folgt indirekt, dass auch alle diese Typen **in jeder Header-Datei bekannt** sind!

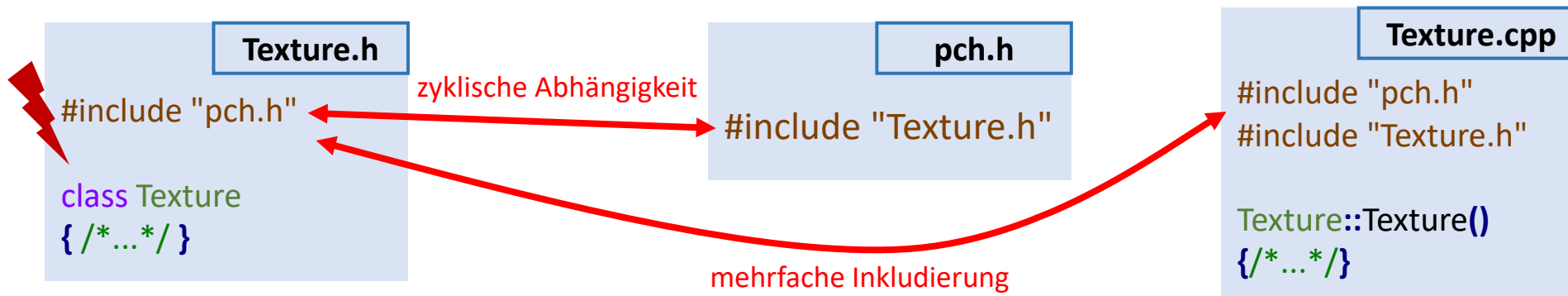


Code-Organisation



Vorkompilierte Header – Andere Header

- Eine andere Header-Datei sollte niemals den vorkompilierten Header inkludieren!
 - Sie kann dann selbst nicht mehr im vorkompilierten Header inkludiert werden (zyklische Abhängigkeit)
 - Sie zwingt den Programmierer bei Inkludierung auch zur Inkludierung des vorkompilierten Headers – vor allem problematisch bei Bibliotheksheadern, die von anderen Projekten inkludiert werden
 - Der vorkompilierte Header wird mehrfach inkludiert (von der .cpp-Datei und dem anderen Header)



Code-Organisation: Vorkompilierte Header



Zusammenfassung

- **Vorkompilierte Header** sind Header-Dateien, die hauptsächlich andere Header-Dateien inkludieren, welche häufig im Programm verwendet werden
 - Meist Bibliotheksheader (<iostream>, <string>, ...)
- Vorkompilierte Header werden vor allen anderen Compiler-Einheiten inkludiert und kompiliert und stehen danach zur Einbindung zur Verfügung
- Vorkompilierte Header **reduzieren die Kompilierzeit** eines Projekts erheblich
 - Reduziert auch die Anzahl inkludierter Header in anderen Headern und Quelldateien
→ geringerer Wartungsaufwand

Code-Organisation

Verständnisfragen

PIN: 886080

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Bidirektionale (ungerichtete) Klassenbeziehungen führen zu zyklischen Abhängigkeiten zwischen Klassendefinitionen.
- Klassen mit zyklischen Abhängigkeiten können problemlos in Programmcode implementiert werden.
- Durch Vorausdeklarationen können Header-Dateien flexibler inkludiert werden.
- Ein Namensraum organisiert Klassen (und andere Definitionen) in einer hierarchischen Struktur, um Namenskollisionen zu vermeiden und den Code besser zu strukturieren.
- Wird ein Typ nicht explizit einem Namensraum untergeordnet, so befindet er sich in keinem Namensraum.
- Die häufige Inkludierung einer Header-Datei hat keinen Einfluss auf die Kompilierzeit.
- Vorkompilierte Header werden in einer eigenen Compiler-Einheit vor allen anderen Compiler-Einheiten kompiliert.
- Vorkompilierte Header sind ein Sprachfeature der Sprache C++.
- Andere Header-Dateien sollten den vorkompilierten Header immer zu Beginn inkludieren.



- **Welche der folgenden Aussagen treffen auf Namensräume zu?**
 - a) Sie bündeln Typnamen in einem eigenen Gültigkeitsbereich.
 - b) Sie verbessern die Wartbarkeit eines Projekts.
 - c) Sie führen zu einer besseren Performanz zur Laufzeit.
 - d) Sie können hierarchisch verkettet werden.
- **Welche der folgenden Operationen sind auf vorausdeklarierten Klassen möglich?**
 - a) Anlegen von Zeigern auf die Klasse.
 - b) Anlegen von Referenzen auf die Klasse.
 - c) Anlegen von Variablen (ohne Zeiger oder Referenz) auf die Klasse.
 - d) Aufruf von Methoden der Klasse.
 - e) Zugriff auf Membervariablen der Klasse.
 - f) Instanziierung von Objekten der Klasse.
 - g) Messung der Größe der Klasse (sizeof()-Funktion).
 - h) Messung der Größe eines Zeigers auf die Klasse (sizeof()-Funktion).